

Reconocimiento de señales con GTSRB

Objetivo. Clasificar imágenes de señales alemanas en 43 categorías y evaluar la respuesta ante nuevas instancias mediante una CNN en Keras.

1. Datos y prevención de fuga de información

GTSRB permite trabajar con grupos reales: cada *track* reúne tomas de una misma señal física. Esta estructura permite separar instancias, en lugar de repartir fotogramas vecinos entre entrenamiento y validación. Se usan las 43 clases oficiales [1]. La unidad de separación es (clase, track), no el fotograma.

Partición	Imágenes	Criterio
Entrenamiento	31.350	1045 tracks
Validación	7.829	261 tracks separados
Test oficial	12.630	Partición oficial completa

Se reserva aproximadamente el 20% de los tracks de cada clase para validación con semilla 42. Las 43 clases están presentes y ningún track cruza train-validación. El manifiesto conserva identificadores, particiones y exclusiones. Esta división por grupos corresponde a la alternativa por lotes de la consigna.

El control de duplicados detectó 8 imágenes de desarrollo idénticas a imágenes del test. Se excluyó el grupo completo al que pertenecían (30 imágenes) después de fijar la partición. La validación y el test conservaron sus imágenes. La limpieza comparó píxeles, sin usar etiquetas ni rendimiento del test. Hubo una evaluación previa a esta corrección; se repitió el protocolo con los mismos hiperparámetros y semillas, sin ajustarlos a ese resultado.

Preprocesamiento compartido: imagen completa suministrada, conversión RGB, redimensionado bilineal a 32 x 32 y división por 255 dentro del modelo. No se aplican aumentos, recortes adicionales ni estadísticas ajustadas con validación o test. La resolución reduce costo de cálculo, aunque puede perder detalles finos.

2. Arquitectura y fundamento matemático

Operación	Salida	Parámetros
Entrada + Rescaling	32 x 32 x 3	0
Conv 3x3, 16 + MaxPool	16 x 16 x 16	448
Conv 3x3, 32 + MaxPool	8 x 8 x 32	4.640
Conv 3x3, 64 + MaxPool	4 x 4 x 64	18.496
Flatten + Dense 64 + Dropout 0,3	64	65.600
Dense 43 + Softmax	43	2.795
Total entrenable		91.979

Convolución 2D. $z[i,j,f] = \text{beta}[f] + \sum(u,v,c) W[u,v,c,f] x[i+u-p,j+v-p,c]$, con $u,v = 0,1,2$; $c = 0,\dots,C-1$ y $p=1$. C es la cantidad de canales de entrada, f identifica uno de los F filtros y $\text{beta}[f]$ es su sesgo. Es la correlación cruzada de Conv2D. El filtro compartido detecta patrones locales con $(3 \cdot 3 \cdot C + 1) \cdot F$ parámetros. Stride 1 y padding *same* conservan la resolución: $n_{\text{out}} = \text{floor}((n+2p-k)/s)+1 = n$, con tamaño n , kernel $k=3$, padding $p=1$ y stride $s=1$ [2].

Pooling. $y[i,j,c] = \max(a,b \text{ en } \{0,1\}) x[2i+a,2j+b,c]$. La ventana 2 x 2 con stride 2 reduce a la mitad cada dimensión. Puede tolerar desplazamientos locales que preserven el máximo; no garantiza invariancia ni equivarianza exacta ante cualquier traslación. Padding y submuestreo introducen efectos de borde y alineación [2,3].

Activaciones y pérdida. $\text{ReLU}(z) = \max(0,z)$ aporta no linealidad sin saturación en el semieje positivo; He usa $\text{Var}(W) = 2/\text{fan_in}$ para conservar escala. Softmax: $p_k = \exp(z_k) / \sum_j \exp(z_j)$. Se minimiza $L = -(1/N) \sum_i \log p(y_i|x_i) + 0,0001 \sum W^2$. N es el tamaño del lote; W incluye kernels convolucionales y de Dense 64, sin sesgos ni capa de salida.

3. Entrenamiento y decisiones de diseño

Los tres bloques 16/32/64 aumentan canales mientras reducen resolución; Dense 64 mantiene pequeña la red. Adam con tasa 0,001 realiza actualizaciones adaptativas; batch 128 equilibra memoria y costo. L2=0,0001 penaliza pesos grandes y Dropout=0,3 dificulta que la red dependa de combinaciones fijas de unidades. Estos valores se mantuvieron en la corrección del dataset; no se hizo búsqueda de hiperparámetros.

El clipping por norma 1 limita cada tensor de gradiente como $g' = g \cdot \min(1, 1/\|g\|)$. Si $\|g\|=0$, $g'=0$. Se mantuvo como precaución del diseño original, sin evidencia de que fuera necesario. Se permiten 30 épocas con early stopping de paciencia 5 sobre pérdida de validación, restaurando los mejores pesos. No se reentrena sobre validación.

Las semillas 42, 43 y 44 cambian inicialización y orden de entrenamiento sobre la misma partición. Se fija de antemano la 42 para entregar; no se elige la mejor corrida. La referencia lineal recibe los mismos píxeles 32 x 32 y una única capa softmax (132.139 parámetros). Compara familias de modelos, sin aislar causalmente una operación.

4. Ajuste, generalización y variabilidad

Modelo / semilla	Época elegida	Acc. train	Acc. val.	F1 val.
Lineal / 42	17	94,27 %	84,97 %	0.8103
CNN / 42	17	99,88 %	95,18 %	0.9308
CNN / 43	26	99,91 %	95,38 %	0.9317
CNN / 44	29	99,89 %	96,09 %	0.9460

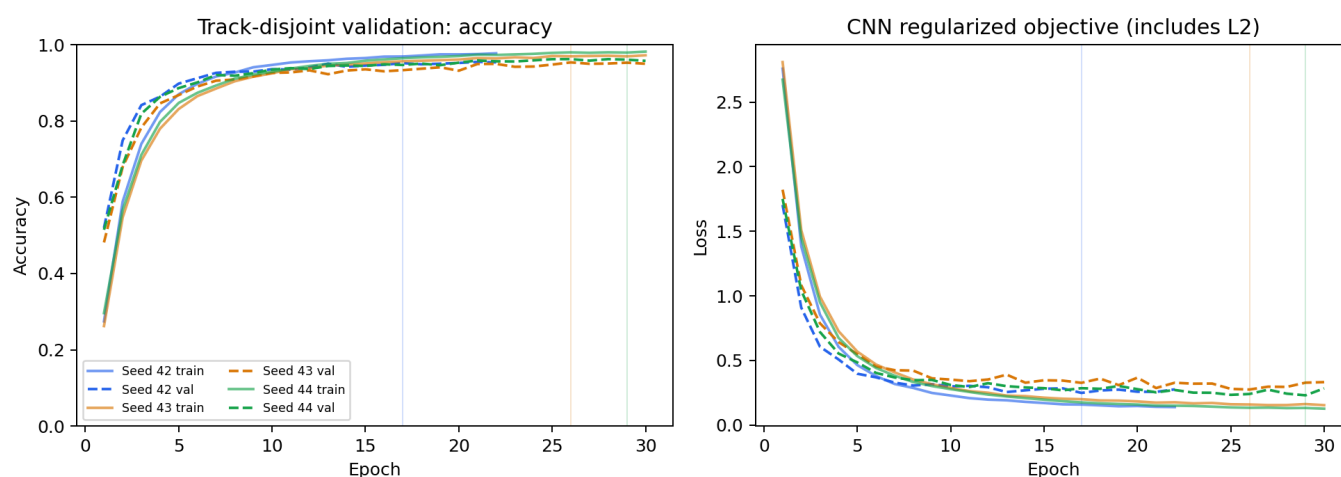


Figura 1. Entrenamiento continuo y validación discontinua, con un color por semilla. En entrenamiento actúa dropout y los pesos cambian durante la época; la tabla reevalúa los pesos restaurados con dropout desactivado. Por eso ambas accuracies de entrenamiento pueden diferir.

La CNN promedia 95,55 % de accuracy de validación, con desviación muestral de 0.48 puntos porcentuales entre semillas. La brecha media train-validación es 4.34 pp. La CNN principal supera al modelo lineal en 10.22 pp.

El error de entrenamiento de la CNN principal es 0,12 %. Su brecha con validación es 4.69 pp: logra ajustar el entrenamiento, pero conserva errores ante otras señales. Esto indica sobreajuste residual. La referencia lineal también muestra una brecha (9.30 pp); su menor rendimiento no se explica sólo por sesgo alto. La comparación cambia arquitectura y regularización, por lo que no aísla el efecto de las convoluciones.

2 de las 3 CNN alcanzaron el máximo de 30 épocas; no se puede asegurar que su pérdida haya convergido. La dispersión entre semillas mide sensibilidad al entrenamiento en esta partición, no variabilidad entre muestras ni una descomposición formal de sesgo y varianza. La validación se usa para elegir la época; el test evalúa el modelo principal fijado de antemano.

5. Evaluación final y análisis de errores

CNN principal / semilla 42	Accuracy	F1 macro	Entropía cruzada
Entrenamiento	99,88 %	0.9985	0.0100
Validación	95,18 %	0.9308	0.1814
Test oficial	92,61 %	0.8969	0.3915

Accuracy mide aciertos por imagen. Las clases tienen frecuencias diferentes: F1 macro promedia las 43 clases con el mismo peso; la pérdida no lleva pesos por clase. La entropía cruzada de la tabla excluye L2, mientras las curvas muestran pérdida regularizada. results.json conserva precisión, recall, F1 y soporte por clase.

Clases con menor F1 en test	Soporte	Recall	F1
27: Pedestrians	60	38,33 %	0.4466
30: Beware of ice/snow	150	58,00 %	0.6850
23: Slippery road	150	88,67 %	0.7644
24: Road narrows on the right	90	66,67 %	0.7792
20: Dangerous curve to the right	90	98,89 %	0.7876

Clase real > predicha (ID)	Errores	% de la clase real
7: Speed limit (100 km/h) > 5: Speed limit (80 km/h)	31	6,89 %
27: Pedestrians > 25: Road work	30	50,00 %
5: Speed limit (80 km/h) > 2: Speed limit (50 km/h)	26	4,13 %
7: Speed limit (100 km/h) > 3: Speed limit (60 km/h)	24	5,33 %

El porcentaje usa todas las imágenes de la clase real como denominador. La matriz completa de 43 x 43 está en test_confusion_matrix.png y results.json. El color expresa la proporción dentro de cada clase real; los números escritos son conteos. Las tablas muestran qué clases concentran los errores.

6. Conclusiones y alcance

La CNN de 91.979 parámetros alcanza 92,61 % de accuracy y 0.8969 de F1 macro en el test oficial. Separar señales físicas evita compartir fotogramas de una misma instancia entre entrenamiento y validación. La limpieza elimina coincidencias exactas con el test. Este control no descarta similitudes visuales ni todos los posibles atajos del dataset.

Se estudia una partición y tres semillas, sin búsqueda de hiperparámetros. No se mide variabilidad entre particiones ni se garantiza desempeño en otros países, cámaras o escenas. El modelo clasifica recortes de señales; no detecta señales en fotografías completas. El archivo exportado incluye reescalado e infer.py repite el redimensionado.

Referencias y reproducción

[1] Stallkamp, J.; Schlipsing, M.; Salmen, J.; Igel, C. (2012). *Man vs. Computer: Benchmarking Machine Learning Algorithms for Traffic Sign Recognition*. Neural Networks, 32, 323-332. [Artículo original](#).

[2] Keras: [Conv2D](#). [3] Keras: [MaxPooling2D](#). Datos: [archivo oficial GTSRB](#). Versiones, hashes y manifiesto acompañan los resultados; README.md detalla la reproducción.